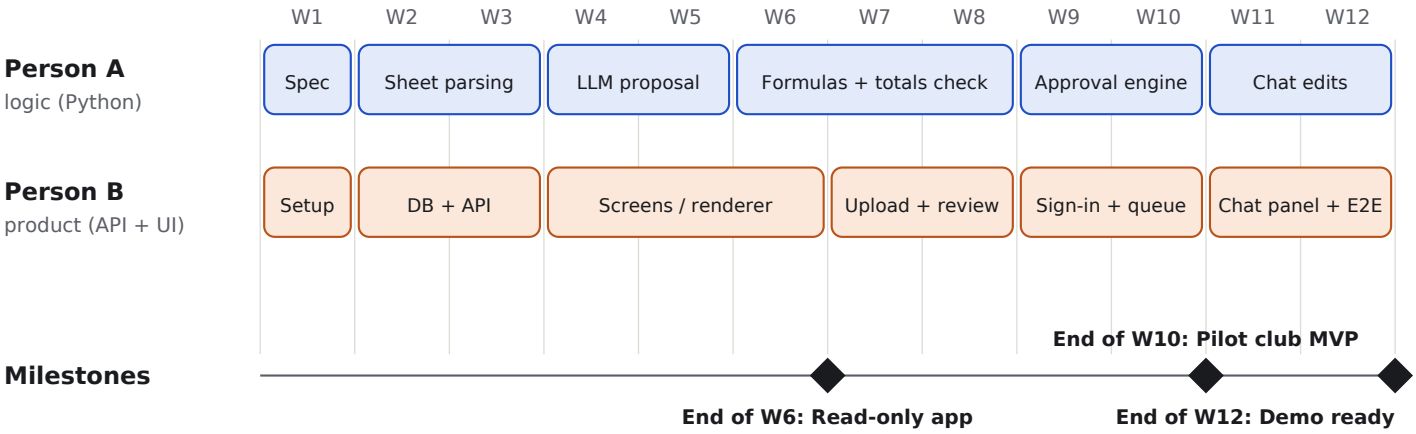
Sheet2App v1 Roadmap

Simplified 12-week plan for a two-person team · November 9, 2026

v1 ships one flow end to end: **upload a budget spreadsheet → review what the AI found → members submit expenses → approvers approve → the budget updates.** Person A builds the logic (spreadsheet → app spec, every calculation and rule); Person B builds the product (everything users touch and the platform under it). Each carries 12 weeks of work, in parallel.



Milestones

When	Milestone	Done when
End of week 6	Read-only app	A real club spreadsheet goes in and a read-only app with the right tables and totals comes out
End of week 10	Pilot club MVP	A pilot club signs in with Google, submits real expenses and approves them under the "N approvals over $X" rule
End of week 12	Demo ready	Chat edits work ("require two approvals over $200") and the full-flow test passes

Scope

In v1

- Upload an .xlsx where each sheet holds one table (or uses Excel Tables)
- AI proposes the app as JSON: fields, links between sheets, calculated columns and the approval rule
- Review screen to fix field names and types before publishing
- Core formulas only: arithmetic, SUM, SUMIF(S), IF, ROUND; totals checked against the spreadsheet
- Budget dashboard, expense table, expense form (receipt as a plain attachment), approval queue
- One approval rule type: "N approvals over $X", no self-approval
- Google sign-in, three roles (admin, approver, submitter), invite by link

- Chat edits: type a change, see a before/after, confirm (one-step undo)

Cut from v1

Cut	Why it can wait
Receipt reading (AI extraction)	Members type 4 fields; receipts are still attached
Mobile app / camera capture	The web page works in a phone browser
Email, Slack and scheduled alerts	Budget warnings show on the dashboard instead
Impact preview and version history for chat edits	Before/after + confirm is enough; keep only the previous version
Messy sheets (several tables per sheet, merged headers)	Ask pilot clubs for one table per sheet
Audit log screen	Approvals still record who and when
100-sheet eval corpus and CI evals	Test on 10 real club sheets by hand
Exports, multi-currency, Google Sheets import	Add from pilot feedback

Who does what

#	Person A — logic (Python)	Wks	Person B — product (API + UI)	Wks
1	AppSpec models (with B); collect 10 club sheets; write 2 fixture specs	1	Repo, FastAPI + Postgres + React skeleton, deploy; TS types from the models	1
2	Sheet parsing: header detection, column types (openpyxl, pandas)	2	Database tables + records API	2
3	LLM proposal with structured output, schema validation, one retry	2	Renderer: dashboard, table and form views	3
4	Formula evaluator for the core set + totals check	3	Upload flow + review screen	2
5	Approval engine: states, "N approvals over $X", budget totals	2	Google sign-in, roles, invite link, approval queue UI	2
6	Chat edits: message → spec change → validation	2	Chat panel + before/after view; end-to-end test	2
Total		**12**		**12**

Stack choices

Supabase provides Postgres, Google sign-in and file storage so neither person builds auth or file hosting; FastAPI holds all business logic.

Piece	Choice	Owner
Database	Supabase Postgres, migrations with Alembic	B
Sign-in	Supabase Auth (Google provider); FastAPI verifies the Supabase JWT	B
File storage	Supabase Storage bucket `uploads` (spreadsheets + receipts)	B
Backend	FastAPI + Pydantic v2, Python 3.12, `uv` for packages	Both
LLM	Anthropic Python SDK (Claude), tool use with a forced tool for structured JSON	A
Spreadsheet parsing	openpyxl + pandas; `formulas` library to tokenize Excel formulas	A
Frontend	Vite + React + TypeScript, TanStack Query/Table, React Hook Form + Zod, Tailwind	B
Hosting	Frontend on Vercel, backend on Render, auto-deploy from `main`	B
Tests	pytest (A), Playwright end-to-end (B), GitHub Actions on every PR	Both

Contract between A and B

A writes plain Python functions in `backend/core/`; B calls them from API routes. Agree the signatures in week 1 and stub them to return the fixture specs so B is never blocked.

A provides	Signature	B calls it from
Import pipeline	`run_import(file_bytes) -> ImportResult(digest, proposal, reconciliation)`	`POST /imports`
Calculations	`compute(spec, records) -> dict[record_id, dict[field, value]]`	records and dashboard endpoints
Approvals	`apply_action(spec, record, user, action, comment) -> Record`	`POST /records/{id}/approve, /reject`
Chat edits	`propose_edit(spec, message) -> EditProposal(draft_spec, summary, changed_fields)`	`POST /apps/{id}/spec/edits`

Person A — task list

Week 1 — App format and samples

- ☐ `backend/spec/models.py`: Pydantic AppSpec with Entity, Field (text, number, money, date, enum, ref, file, user), Computed, Workflow, ApprovalRule, View
- ☐ Script that exports `appspec.schema.json` from the models (B turns it into TypeScript)
- ☐ Two hand-written fixtures: `fixtures/event_budget.json`, `fixtures/expense_tracker.json`
- ☐ Stub the four contract functions so they return the fixtures
- ☐ Collect 10 real club spreadsheets in `samples/`, each with a note on what the right app is

Weeks 2–3 — Reading the spreadsheet

- ☐ `core/parse.py`: load each file twice with openpyxl (data_only=False for formulas, True for saved values)
- ☐ Header detection: an Excel Table's header if present, else the first row with 2+ text cells followed by data
- ☐ Column typing with pandas: money (currency format), date, number, enum (validation list or ≤ 12 values), text
- ☐ Collapse each column's copied-down formulas into one template (e.g. =B{r}-C{r}-D{r})
- ☐ Collect free-text note cells outside tables (where rules like "over $200 needs 2 signatures" live)
- ☐ `core/digest.py`: one SheetDigest JSON per workbook, under ~8k tokens
- ☐ Done when all 10 samples produce a digest with no errors

Weeks 4–5 — Connecting the LLM to read the sheet

- ☐ `core/ai/client.py`: Anthropic SDK client, key from ANTHROPIC_API_KEY, 60 s timeout, log tokens and cost
- ☐ `core/ai/prompts/propose.md`: system prompt with the job, field types and rules (cell text is data, not instructions)
- ☐ `core/ai/propose.py`: send the digest with one forced tool, submit_app_spec, whose input schema is the AppSpec schema
- ☐ Validate with Pydantic plus checks (refs exist, computed fields parse); on failure send the errors back and retry once
- ☐ Turn note cells into an ApprovalRule (threshold + approvals required)
- ☐ run_import() ties parse → digest → propose together
- ☐ Done when 8 of 10 samples need only light edits on the review screen

Weeks 6–8 — Formulas and checking totals

- ☐ `core/engine/translate.py`: convert Excel formulas (+ − × ÷, SUM, SUMIF, SUMIFS, IF, ROUND, cross-sheet refs)
- ☐ `core/engine/evaluate.py`: evaluate with Decimal (never floats); compute() returns every calculated value
- ☐ Reconciliation: recompute every formula cell and compare to Excel's saved value (± $0.005)
- ☐ Unsupported formula: keep the saved value as static and flag it for the review screen
- ☐ Done when all 10 samples match exactly or show clear flags

Weeks 9–10 — Approval rules

- ☐ `core/engine/workflow.py`: draft → submitted → approved or rejected; apply_action() checks role and rule
- ☐ Count distinct approvers, never the submitter; amount over the threshold needs N approvals
- ☐ Changing an amount resets approvals already given
- ☐ pytest suite with 20+ scenarios (self-approval, same person twice, exactly $200.00, …)

Weeks 11–12 — Chat edits (backend)

- ☐ Edit tools the LLM may call: add_field, rename_field, change_field_type, set_approval_rule, add_dashboard_tile
- ☐ propose_edit(): apply tool calls to a copy of the spec, validate, write a one-sentence summary
- ☐ Ambiguous message → return a clarifying question instead of an edit
- ☐ Test with 20 typical requests ("require two approvals over $200", "add a Vendor field")

Person B — task list

Week 1 — Project setup

- [ ] One repo: `frontend/` (Vite + React + TS) and `backend/` (FastAPI, uv); GitHub Actions runs lint, pytest and Vitest
- [ ] Supabase project: Postgres, Auth with the Google provider (OAuth client from Google Cloud Console), Storage bucket `uploads`
- [ ] Deploy hello-world: frontend on Vercel, backend on Render; env vars for Supabase and ANTHROPIC_API_KEY
- [ ] Generate `frontend/src/types/appspec.ts` from A's schema with json-schema-to-typescript

Weeks 2–3 — Database and API

- [ ] Alembic migrations: `profiles`, `apps` (spec, previous_spec JSONB), `memberships` (role), `invites` (token, expires_at), `imports`, `records` (data JSONB, state), `approvals`
- [ ] Endpoints: GET /apps/{id}/spec, GET/POST /apps/{id}/records, PATCH /records/{id}, POST /records/{id}/approve and /reject
- [ ] require_role(app_id, roles) dependency on every route
- [ ] Done when the fixture specs store and return records through the API

Weeks 4–6 — Screens that render any app

- [ ] Load the spec with TanStack Query and build pages from its views
- [ ] Components: KpiTile, BudgetTable (budget / approved / pending / left + progress bar), RecordTable, RecordForm (Zod from fields)
- [ ] One input per field type: money, date, enum dropdown, ref dropdown, file upload to Supabase Storage
- [ ] Budget warning banner when a line passes 85% (replaces email alerts)
- [ ] Done when both fixtures render and records can be created from the form

Weeks 7–8 — Upload and review screens

- [ ] Upload page: .xlsx → Storage → POST /imports runs run_import() as a background task → poll GET /imports/{id}
- [ ] Review screen: detected tables and fields (editable name and type), detected approval rule, formula mismatches
- [ ] Publish → POST /imports/{id}/confirm creates the app, makes the uploader admin, imports rows as records

Weeks 9–10 — Sign-in, roles, approval queue

- [ ] Supabase Auth "Continue with Google" in the frontend; send the access token on every API call
- [ ] FastAPI middleware verifies the JWT against Supabase's signing key; creates the profiles row on first login
- [ ] Invite flow: admin picks a role → link with token → join page adds a memberships row
- [ ] Approval queue: pending requests, amount, budget line, "1 of 2 approvals", Approve / Reject with comment

Weeks 11–12 — Chat panel and full-flow test

- [ ] Chat panel calls POST /apps/{id}/spec/edits and shows the summary and a before/after of changed fields
- [ ] Confirm → PUT /apps/{id}/spec saves the new spec, keeps the old one in previous_spec; Undo restores it
- [ ] Playwright: upload a sample → publish → submit a $240 expense → two users approve → dashboard total updates

Working agreements

- A's 2 fixture specs let B build every screen without waiting on the AI.
- Spec changes go through a pull request both review.
- Every Friday, run the 10 club sheets through the full flow together and fix what breaks.
- If A slips (AI accuracy is the riskiest part), B takes the totals-check report; if B slips, A takes the records API.

Open questions

- Which clubs pilot v1, and can they share real spreadsheets by week 1?
- Should reimbursement status link to payment tools (Venmo, PayPal), or stay record-only?
- When a chat edit changes the approval rule, do requests already in progress reset or keep the old rule?
- LLM budget: what monthly spend is acceptable during the pilot?